DroPE: что удалось повторить на двух T4

Я повторил часть экспериментов из статьи Extending the Context of Pretrained LLMs by Dropping Their Positional Embeddings (Gelberg et al., 2025). Для обучения использовал Kaggle с двумя T4. Времени хватило на 0.3062 млрд входных токенов. У авторов самый короткий сопоставимый запуск использует 31.46 млрд.

Результат получился смешанным. После удаления RoPE модель заметно улучшилась за время дообучения, но до исходного качества не дошла. На длинном контексте DroPE обошёл обычную RoPE без масштабирования, однако уступил YaRN и Dynamic NTK. Найти спрятанное число в тексте DroPE не смог даже в коротком тесте.

1. Идея статьи

RoPE помогает attention учитывать взаимное положение токенов. Во время обучения это полезно: модели проще выучить зависимости в тексте. Но на последовательностях длиннее обучающих позиционные зависимости меняются. Качество может резко упасть.

Авторы предлагают убрать RoPE из уже обученной модели и дать ей немного дообучиться на привычной длине. Этот этап они называют recalibration. Длинные тексты для него не нужны. Модель уже умеет работать с языком, ей остаётся приспособиться к изменённому attention.

Для SmolLM авторы используют QKNoPE. Позиционное вращение отключается, а query и key нормируются по RMS. Причинная маска остаётся: токен по-прежнему видит только предшествующий текст. Поэтому удаление явной позиционной кодировки не стирает всю информацию о порядке.

На длинном контексте attention распределяет вероятность между большим числом ключей. Авторы компенсируют это коэффициентом beta, который умножает attention logits. Для SmolLM используется формула **beta(L) = 1 + 0.103 ln(max(1, L/2048))**. Она меняет масштаб attention. Temperature генерации остаётся отдельной настройкой.

В статье есть два типа экспериментов. В контролируемом обучении с нуля DroPE после 14 тысяч шагов RoPE и 2 тысяч шагов адаптации догоняет RoPE, обученную 16 тысяч шагов. Это результат авторов из Figure 2. В другой части статьи они адаптируют готовые модели.

Моя работа ближе ко второму сценарию: **SmolLM-360M из §5.1, Appendix C.1 и Table 4**. Авторы проверяют бюджеты 31.46, 62.9 и 125.8 млрд токенов. В тексте статьи они округлены до 30, 60 и 120 млрд. У меня получилось около 0.97% от самого малого бюджета. Я повторяю процедуру адаптации на меньшем бюджете. Прямо сравнивать эти кривые с авторскими нельзя.

2. Как я настроил эксперимент

Настройка	В статье	В моём запуске
Модель	Готовая SmolLM-360M	HuggingFaceTB/SmolLM-360M
Длина при адаптации	2048	2048
Данные	FineWeb-Edu	PrimeIntellect/fineweb-edu
Бюджет	31.46–125.8 млрд токенов	0.3062 млрд входных токенов
Общий batch	512 последовательностей	8: две GPU, microbatch 1, accum 4
Максимальный LR	1e-3	3e-5
Изменение LR	Warmup 490, затем cosine	Warmup 30, затем постоянный LR
AdamW	beta=(.9,.95), weight decay=.1	beta=(.9,.999), eps=1e-8, weight decay=0
Точность	bf16	fp16, ZeRO stage 1, SDPA
Текст для NIAH	Paul Graham	FineWeb, небольшая выборка

Настройки отличаются сразу в нескольких местах. Одинаковой длины и названия модели для точного повторения недостаточно. Предварительный замер скорости выбрал microbatch=1 без gradient checkpointing. Microbatch=4 без checkpointing вызвал OOM. На T4 использовался fp16. Triton и torch.compile отдельно не проверялись.

Для проверки качества я оставил 16 блоков по 2048 токенов на dev и 8 блоков по 8192 на test. Они взяты из отдельных физических шардов: 11987 и 11986 соответственно. Тесты на 2048 и 4096 используют начало тех же test-блоков. SHA256 файла test.npy совпадает в оценках до и после продолжения обучения.

Обучающие документы делятся между двумя процессами. Текст токенизируется с EOS и собирается в блоки без padding. Совпадающих хешей документов между использованными train, dev и test не нашли. Это проверка точных совпадений, а не полная семантическая дедупликация. Два dev-блока длиной 8192 использовались для подбора beta. Test в подборе не участвовал.

3. Как прошло обучение

Первый этап закончился на шаге 8712. Успешных обновлений весов было 8700, ещё 12 fp16-обновлений пропущены. Модель обработала 142 737 408 входных токенов. Затем я продолжил обучение с сохранёнными состояниями optimizer, scaler, RNG и потока данных.

Второй этап добавил 9977 шагов и дошёл до **step 18689**. Из них 9965 обновлений прошли успешно, 12 пропущены. Итого: 18 665 успешных обновлений, 24 пропуска и **306 200 576 входных токенов**. Счётчик включает батчи пропущенных обновлений, поэтому все эти токены нельзя считать токенами успешного изменения весов.

Средний шаг занимал 3.412 с на первом этапе и 3.007 с на втором. Пиковый объём памяти, выделенной PyTorch, был около 9.25 GiB на GPU. В это число не входит вся память процесса. Подготовка продолжения заняла примерно 16.6 минуты: код заново воспроизводил поток и сверял каждый уже закэшированный блок. В переносимой версии снимок буфера копируется, чтобы избежать aliasing. Порядок данных старого запуска подтвердился при replay.

Во всех сохранённых шагах train loss оставался конечным. На втором этапе dev CE снизилась с 3.8254 до 3.4976, а dev PPL с 45.85 до 33.04. Это один запуск. Проверки на нескольких seeds не было.



Ошибка на dev во втором этапе. Источник: results/stage2/dev_curve.json.

4. Качество языковой модели

Здесь CE обозначает среднюю ошибку предсказания следующего токена, а PPL = exp(CE). Чем меньше значение, тем лучше. Для блока длиной L оцениваются L−1 следующих токенов. Каждое число в итоговой таблице получено на восьми test-блоках. Эти блоки и их префиксы связаны между собой, поэтому считать все измерения независимыми нельзя.

Модель или настройка	PPL 2048	PPL 4096	PPL 8192
RoPE без масштабирования	12.64	76.91	286.88
Сразу после удаления RoPE	612.54	нет замера	нет замера
DroPE, шаг 8712, beta с dev	42.66	56.79	85.86
DroPE, шаг 18689, beta с dev	30.95	41.23	81.02
RoPE + YaRN	нет замера	12.08	12.11
RoPE + Dynamic NTK	нет замера	12.46	19.26

Главное улучшение видно в исходном контексте 2048: после удаления RoPE PPL была 612.54, после обучения стала 30.95. Но у исходной модели она равна 12.64. Разница всё ещё в 2.45 раза.

Между двумя этапами PPL снизилась на 27.4% в контексте 2048, на 27.4% в 4096 и на 5.6% в 8192. Последнее число отражает сразу два изменения: продолжение обучения и новую beta. Отдельно их эффекты здесь не измерялись.

Зачем понадобился подбор beta

Для финальной модели dev выбрал beta=1.25 на 4096 и beta=1.5 на 8192. У checkpoint 8712 на обеих длинах была beta=1.25. Проверялась сетка {1, beta из статьи, 1.25, 1.5, 1.75, 2} на двух dev-блоках. Это маленькая выборка. Настройка, которая победила на ней, может оказаться хуже на другом тексте.

Без масштабирования финальный DroPE на 8192 даёт PPL около 16 119. Формула из статьи снижает её до 1387, а beta с dev до 81.02. В этом запуске одного отключения RoPE для длинного контекста явно недостаточно.

YaRN и Dynamic NTK применялись к исходной RoPE без дообучения. Оба метода лучше DroPE на нашем LM-тесте. При этом отдельного RoPE-дообучения на тех же 0.3062 млрд токенов не было. Поэтому влияние дополнительных данных от влияния изменения attention полностью не отделено.

Что нужно учитывать в этих числах

Основные RoPE/DroPE-метрики сохранились в JSON/CSV. Результаты YaRN/NTK восстановлены из консольного вывода законченного запуска: CE перенесена по каждому блоку, затем средняя CE и PPL пересчитаны. Архив этого запуска потерян после отключения сессии.

YaRN выдал предупреждение: max_position_embeddings остался 2048, хотя явный factor был 2 или 4. Hugging Face сообщил, что использует этот factor. В новой версии скрипта метаданные согласованы с длиной, но повторного GPU-замера после правки не было. Для Dynamic NTK исходный предел 2048 сохранён намеренно.

Итоговые tail512-метрики используют последние 512 предсказываемых токенов. В первоначальном ночном evaluator срез захватывал 511. Для основных сравнений использована отдельная исправленная оценка из evaluation/worker.py.

5. Может ли модель найти число в тексте

NIAH проверяет, найдёт ли модель факт среди другого текста. В моём варианте в FineWeb-текст вставляется семизначное число с ключом, а в конце модель просят его назвать. Генерация жадная, максимум 24 новых токена. Сравнивается первое извлечённое семизначное число, а не вся строка ответа.

Два шаблона ключа чередуются. Текст выбирается из восьми test-блоков, seed=20261010+sample. Поэтому примеры частично повторяются и связаны друг с другом. Их немного, но уже короткий контроль помогает увидеть проблему.

Условие	RoPE	Финальный DroPE
512 токенов, глубина .9	10/12 (83.3%)	0/12
2048, глубина .1	3/8 (37.5%)	0/8
2048, глубина .5	4/8 (50.0%)	0/8
2048, глубина .9	8/8 (100%)	0/8
Всего на 2048	15/24 (62.5%)	0/24

RoPE проходит большинство коротких примеров. DroPE не проходит ни одного. На маленьком сохранённом тесте с двумя примерами на ячейку картина была такой же. Улучшение LM-loss пока не превратилось в способность находить нужный факт.

Исправление KV-cache

До этих проверок нашлась ошибка совместимости: авторский attention ожидал аргумент past_key_value, а установленный Transformers передавал past_key_values. Адаптер перенаправляет аргумент. Веса модели при этом не меняются.

У финального checkpoint среднее расхождение logits между вычислением с кэшем и без него снизилось примерно с 1.503 до 0.0173. Разница CE между SDPA и eager составила около 7.3e−5. Проверки причинности, кэша и backend не выставили флаг suspect. Это полезная диагностика, но она не доказывает отсутствие всех ошибок. Обучение шло с use_cache=False, поэтому найденная проблема касалась генерации с кэшем.

Ограничения теста

Дополнительный запуск завершился без ошибок в обеих ролях, обе вывели INTEGRITY OK. Но подробные численные проверки и тексты ответов вместе с архивом потеряны. Их нельзя восстановить из итоговых counts. В комплекте есть только то, что осталось в консоли.

Без спрятанного факта запускалось по четыре контроля. Значение False в таком случае нельзя читать как обычную ошибку accuracy: нужного факта в тексте нет. Подробные ответы этих контролей тоже недоступны.

Запрос длиной 2048 вместе с продолжением может выйти за исходный предел ещё на 24 токена. Это оговорка к основному тесту. Но DroPE провалил и контроль 512, поэтому одной этой причиной результат не объяснить. Авторский checkpoint через наш evaluator не проверялся. Retrieval для YaRN и NTK тоже не измерялся.

6. Что получилось и что осталось проверить

Модель смогла частично восстановить языковое качество после удаления RoPE. Исходного качества на 2048 она не достигла. На длинном LM-тесте помог подбор beta, но YaRN и Dynamic NTK на том же тесте лучше. Поиск факта в тексте у DroPE не восстановился даже в коротком контроле.

Это не повод считать статью опровергнутой. Мой бюджет меньше самого малого сопоставимого бюджета авторов примерно в 103 раза. Кроме объёма обучения, отличаются batch, optimizer и schedule. Проверочные выборки тоже небольшие.

Если продолжать работу, сначала стоит пропустить авторский checkpoint через наш evaluator. Затем добавить RoPE, дообученную на таком же числе токенов, и повторить оценку на нескольких seeds с более разнообразным NIAH. Более длинная адаптация поможет проверить, достаточно ли текущей модели времени на восстановление. Эти эксперименты пока не выполнены.

Источники

1. Gelberg et al. Статья v1: https://arxiv.org/abs/2512.12167

2. Код авторов:
https://github.com/SakanaAI/DroPE/tree/260a8bb9f4fda0673266ddfee7668915aade8b33

3. RoPE scaling в Transformers 4.56.2:
https://github.com/huggingface/transformers/blob/v4.56.2/src/transformers/modeling_rope_utils.py

4. Собственные результаты: results/stage1/, results/stage2/, results/evaluation/ и results/extra/.